

P1010R1

Container support for implicit lifetime types

Draft Proposal, 2018-10-08

This version:

<http://wg21.link/p1010r1>

Authors:

[Mark Zeren](#) (VMware)

[Chris Kennelly](#) (Google)

Audience:

LEWG, LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

<https://github.com/mzeren-vmw/iso/blob/master/resize/p1010r1.bs>

Abstract

Extend allocators, `allocator_traits`, and `vector` to allow efficient addition of elements of *implicit lifetime* type.

Table of Contents

1 Summary

1.1 Approach

2 Revision History

2.1 R0 → R1

3 Motivating examples

3.1 Example: reading from the network

3.2 Example: stamping a pattern

4 Related work

4.1 Google `basic_string`

4.2 Boost containers

4.3 MongoDB

4.4 VMware string builders

4.5 Discussion on std-proposals

4.6 P1020R0

5 Alternatives Considered

5.1 Free function `std::implicit_construct`

5.2 `insert_from_capacity`

5.3 `implicit_construct`

5.4 `uninitialized_data`

6 Open Issues

7 Wording

7.1 `[allocator.requirements]`

7.2 `[allocator.traits]`

7.3 `[container.requirements.general]`

7.4 `[vector]`

8 Acknowledgments

References

Informative References



... there is a family of types for which programmers
assume they do not need to explicitly create objects ...

[\[P0593R2\]](#)



§ 1. Summary

[\[P0593R2\]](#) Gives us defined behavior for creating objects from raw, application initialized, memory. These *implicit lifetime* types include:

- Scalar types
- Array types (with any element type)
- Class types with a trivial destructor and a trivial constructor (of any kind)

When working with implicit lifetime types applications may be able to bulk initialize objects without value initialization (zeroing) or explicit calls to constructors. For example, the application may read objects from the network, or it may stamp out a pattern of objects, etc.. This proposal enables these optimizations for allocator aware containers, and extends `vector` to allow them. Prior work in this area has relied on trivial default construction. However, [\[P0593R2\]](#) allows class types with *non-trivial* default constructors that have trivial move or copy constructors to be implicit lifetime types. This proposal includes this alternate set of types. Default construction typically leaves indeterminate values in elements controlled by the `vector`, exposing it to undefined behavior until the application initializes all such elements. This proposal avoids this "trap state" for the container by not incorporating elements into the container until they have been initialized.

We have proposed related changes to `basic_string` in a separate paper [\[P1072R1\]](#).

§ 1.1. Approach

Allocator aware containers must cooperate with their allocator for object construction and destruction. Working with the broader set of implicit lifetime types requires that the container use a two step interaction with the application. First, the container exposes memory that the application initializes. Second, the application tells the container how many objects were initialized. The container can then tell the allocator about the newly created objects.

References and wording are relative to [\[N4762\]](#).

Starting with `[allocator.requirements]` (Table 33), we add:

- `a.implicit_construct(c)` - This expression informs the allocator *post facto* of an object of implicit lifetime type that has been initialized and implicitly created by the application. This member function, if provided, does not participate in overload resolution unless `C` is an implicit lifetime type. By default it does nothing.

Then in `[allocator_traits]` we add a new *optional* member:

- `implicit_construct(Alloc& a, T* p)` - This member function:
 - Calls `a.implicit_construct(p)` if it is well-formed, otherwise ...
 - Does nothing if `T` is an implicit lifetime type and `a.construct(p)` is *not* well-formed, otherwise ...
 - Does not participate in overload resolution.

(The intent is to leave the meaning of allocators which define `construct(T*)` unchanged, but to allow those that don't, including the default allocator, to support `implicit_construct` implicitly.)

Finally, in `[vector]` we add two member functions:

- `T* uninitialized_data()` - Returns a pointer to storage that would back elements `[size(), capacity())`. Note that this storage may contain indeterminate values (`[dcl.init]`p12). The application may initialize the memory by casting to `void*` and then to `char*`, `unsigned char*`, or `std::byte*`. (`[basic.life]`p6.4).

Does not participate in overload resolution if

`allocator_traits<Allocator>::rebind_traits<U>::implicit_construct(U*)` is not well-formed.

- `insert_from_capacity(size_type n)` - Appends `n` elements from capacity. The application must have initialized the storage backing these elements otherwise the behavior is undefined.

Does not participate in overload resolution if

`allocator_traits<Allocator>::rebind_traits<U>::implicit_construct(U*)` is not well-formed.

§ 2. Revision History

§ 2.1. R0 → R1

- Rebased on [\[N4762\]](#).
- Updated reference to [\[P1072\]](#).

§ 3. Motivating examples

The extra overhead described in the examples below is often small, yet the optimization can be significant in performance critical execution paths.

Casts, using declarations, and other details have been elided to keep the examples simple.

§ 3.1. Example: reading from the network

The current `vector` interface forces a copy when reading objects from the network, or a file, etc.. (`std::byte` keeps the example simple, but the principle applies to user defined implicit lifetime types as well.):

```
using ByteVec = vector<byte>;

class Socket {
public:
    size_t Read(byte* buf, size_t size);
    ...
};

unsigned ReadSome(ByteVec* out, Socket& socket)
{
    byte buf[kBufferSize];
    auto size = socket.Read(&buf[0], kBufferSize);
    out->insert(out.end(), &buf[0], &buf[0] + size); // BAD: Copies.
    return size;
}
```

With the changes proposed in this paper, the above example would be optimized as:

```

unsigned ReadSome(ByteVec* out, Socket& socket)
{
    out.reserve(out.size() + kBufferSize);
    auto size = socket.Read(out->uninitialized_data(),    // GOOD: No copies.
                           data.capacity() - data.size());
    out->insert_from_capacity(size);                        // GOOD: No-op.
    return size;
}

```

Note: `reserve` bypasses `vector`'s normal growth algorithm which can lead to unnecessary allocations. Resolving that issue is out of scope for this proposal.

§ 3.2. Example: stamping a pattern

For a second example, consider stamping a repeating pattern of elements. `vector`'s interface offers two options, neither optimal:

1. Call `resize` and write directly into the container. However, this *value initializes* elements, typically writing zeros:

```

using IntVec = vector<int>;

void AppendPattern(IntVec& out, span<const int> pattern, unsigned count)
{
    auto start = out.size();
    auto step = pattern.size();
    out.resize(start + step * count);    // BAD: Zeros.
    for (auto cur = out.begin() + start;
         cur < out.end(); cur += step) {
        memcpy(&*cur, pattern.data(),    // GOOD: No bookkeeping.
              step * sizeof(int));
    }
}

```

2. Call `reserve` and then `insert` in a loop. However, this incurs bookkeeping overhead in each insert:

```

void AppendPattern(IntVec& out, span<const int> pattern, unsigned count
{
    out.reserve(out.size() + pattern.size() * count); // GOOD: No zeros.
    for (unsigned i = 0; i < count; ++i) {
        out.insert(out.end(), pattern.begin(),          // BAD: Bookkeeping
                  pattern.end());
    }
}

```

With the changes proposed in this paper the above example would be optimized as:

```

void AppendPattern(IntVec& out, span<const int> pattern, unsigned count
{
    auto step = pattern.size();
    auto total = step * count;
    out.reserve(out.size() + total); // GOOD: No zeros.
    int* cur = out.uninitialized_data();
    int* end = cur + total;
    for (; cur < end; cur += step) {
        memcpy(cur, pattern.data(), // GOOD: No bookkeeping.
              step * sizeof(int));
    }
    out.insert_from_capacity(total); // GOOD: No-op.
}

```

§ 4. Related work

As mentioned above, all related work to date has used default initialization. This is the first proposal that uses implicit lifetime types.

§ 4.1. Google `basic_string`

Google has hacked their internal `basic_string` implementation to provide a related `resize_uninitialized` API. They have measured performance improvements (that are not public) that justify maintaining this extension.

Google's Abseil open source library provides hooks for other users that want to independently apply the same hack. See: https://github.com/abseil/abseil-cpp/blob/master/absl/strings/internal/resize_uninitialized.h

Google's Protocol Buffers open source library takes advantage of Abseil's hooks to improve performance. See:

https://github.com/google/protobuf/blob/master/src/google/protobuf/stubs/stl_util.h#L61

The Snappy compression library has hooks for a similar hack:

<https://github.com/google/snappy/search?q=STLStringResizeUninitialized>

Tensor Flow does too:

<https://github.com/tensorflow/tensorflow/search?q=STLStringResizeUninitialized>

§ 4.2. Boost containers

Boost provides a related optimization for vector-like containers, introduced in [\[SVN r85964\]](#) by Ion Gaztañaga.

```
Default initialization for vector-like containers
Complexity guarantees for associative container constructors and ordered
Added benchmark for associative containers
Fixes #9166
```

E.g.: [boost/container/vector.hpp](#):

```

///! <b>Effects</b>: Constructs a vector that will use a copy of allocator a
///! and inserts n default initialized values.
///!
///! <b>Throws</b>: If allocator_type's allocation
///! throws or T's default initialization throws.
///!
///! <b>Complexity</b>: Linear to n.
///!
///! <b>Note</b>: Non-standard extension
vector(size_type n, default_init_t);
vector(size_type n, default_init_t, const allocator_type &a)
...
void resize(size_type new_size, default_init_t);
...

```

These optimizations are also supported in Boost Container's `small_vector`, `static_vector`, `deque`, `stable_vector`, and `string`.

§ 4.3. MongoDB

MongoDB has a string builder that could have been implemented in terms of `basic_string` as a return value. However, as explained by Mathias Stearn, zero initialization was measured and was too costly. Instead a custom string builder type is used:

E.g.: <https://github.com/mongodb/mongo/blob/master/src/mongo/db/fts/unicode/string.h>

```

/**
 * Strips diacritics and case-folds the utf8 input string, as needed to sup
 * options.
 *
 * The options field specifies what operations to *skip*, so kCaseSensitive
 * means to skip case folding and kDiacriticSensitive means to skip diacrit
 * stripping. If both flags are specified, the input utf8 StringData is retu
 * directly without any processing or copying.
 *
 * If processing is performed, the returned StringData will be placed in bu
 * buffer's contents (if any) will be replaced. Since we may return the inp
 * unmodified the returned StringData's lifetime is the shorter of the inpu
 * utf8 and the next modification to buffer. The input utf8 must not point
 * buffer.
 */
static StringData caseFoldAndStripDiacritics(StackBufBuilder* buffer,
                                             StringData utf8,
                                             SubstrMatchOptions options,
                                             CaseFoldMode mode);

```

(Comments re-wrapped.)

§ 4.4. VMware string builders

VMware has an internal string builder implementation that avoids `std::string` due, in part, to `reserve`'s zero-writing behavior. This is similar in spirit to the MongoDB example above.

§ 4.5. Discussion on std-proposals

This topic was discussed in 2013 on std-proposals in a thread titled "Add `basic_string::resize_uninitialized` (or a similar mechanism)":
<https://groups.google.com/a/isocpp.org/forum/#!topic/std-proposals/XIO4KbBTxl0>

§ 4.6. P1020R0

See also [\[P1020R0\]](#) "Smart pointer creation functions for default initialization".

§ 5. Alternatives Considered

§ 5.1. Free function `std::implicit_construct`

This proposal adds an optional member to `std::allocator_traits`. In [P0653R2], which added the optional member `std::pointer_traits::to_address`, the author also added a free function `std::to_address`. The free function provides uniform access to user specializations of `pointer_traits` that do not define `to_address`. A free function is not necessary or applicable here because containers need to directly test for well-formedness of `allocator_traits<Allocator>::rebind_traits<U>::implicit_construct(U*)` to correctly enable / disable methods in their interface.

Also consider that `allocator_traits` is designed to provide uniform operations over custom allocators. [allocator.requirements]p2 does allow for user specialization of `allocator_traits`, but do users need this additional customization point? Should `allocator_traits` have been named `allocator_adapter`? Should we discourage or deprecate user specialization?

§ 5.2. `insert_from_capacity`

What name and return type should we use?

Return type	Name	Comments
<code>iterator</code>	<code>insert_from_capacity(n)</code>	as proposed
<code>vector&</code>	<code>append_from_capacity(n)</code>	like <code>basic_string::append</code> . <code>vector</code> does not have <code>append</code> .
<code>void</code>	<code>resize_uninitialized(new_size)</code>	but the elements <i>are</i> initialized
<code>void</code>	<code>resize_from_capacity(new_size)</code>	<code>resize</code> implies / is associated with reallocation, but this operation cannot reallocate
<code>void</code>	<code>extend(n)</code> , <code>grow(n)</code> , etc.	some new term...

§ 5.3. `implicit_construct`

What name should we use?

Return type	Name	Comments
-------------	------	----------

<code>void</code>	<code>implicit_construct(T *)</code>	as proposed
<code>void</code>	<code>bless(T *)</code>	From [P0593R2] .

§ 5.4. `uninitialized_data`

What name and return type should we use?

Return type	Name	Comments
<code>T*</code>	<code>uninitialized_data()</code>	as proposed
<code>span<T></code>	<code>uninitialized_data()</code>	Nevin commented: the current proposal makes <code>capacity()</code> a salient property of <code>vector</code> and that that may be unsirable from the LEWG point of view. Mathias suggested returning <code>span<T></code> .

§ 6. Open Issues

- **Q:** Can `implicit_construct` throw?

A: Yes.

- It would be useful to add a discussion of the tradeoffs between implicit lifetime types and default initialization.

§ 7. Wording

§ 7.1. `[allocator.requirements]`

In `[allocator.requirements]` Table 33 add the `a.implicit_construct(c)` expression:

Expression	Return type	Assertion/note pre-/post- condition	Default
...			
<code>a.construct(c, args)</code>	(not used)	<i>Effects:</i> Constructs an object of type C at c	<code>::new((void*)c)</code> <code>C(forward(args)...)()</code>
<u><code>a.implicit_construct(c)</code></u>	(not used)	<u><i>Effects:</i> Informs the allocator that an object of type C has been implicitly created at c. Only participates in overload resolution if C is an implicit lifetime type.</u>	<u>Does nothing</u>
<code>a.destroy(c)</code>	(not used)	<i>Effects:</i> Destroys the object at c	<code>c->~C()</code>

...

And then in [allocator.requirements]p9 add references to `implicit_construct`:

9. An allocator may constrain the types on which it can be instantiated and the arguments for which its `construct`, `implicit_construct`, or `destroy` members may be called. If a type cannot be used with a particular allocator, the allocator class or the call to `construct`, `implicit_construct`, or `destroy` may fail to instantiate.

§ 7.2. [allocator.traits]

Note: Following the pattern of `pointer_traits::to_address` we do not add `implicit_construct` to the synopsis in [allocator.traits]. Placing it there would make it a required member of `allocator_traits` which would break existing user specializations.

Add a new § 19.10.9.3:

19.10.9.3 Allocator traits optional static member functions [allocator.traits.optmem]

```
template<class T, class... Args>  
static void implicit_construct(Alloc& a, T* p);
```

1. *Effects:* Calls `a.implicit_construct(p)` if that call is well-formed; otherwise, does nothing if `T` is an *implicit lifetime* type and `a.construct(p)` is *not* well-formed; otherwise, does not participate in overload resolution.

§ 7.3. [container.requirements.general]

In [container.requirements.general]p3 add references to `implicit_construct`:

3. For the components affected by this subclause that declare an `allocator_type`, objects stored in these components shall be constructed using the function `allocator_traits<allocator_type>::rebind_traits<U>::construct` or `allocator_traits<allocator_type>::rebind_traits<U>::implicit_construct` and destroyed using the function `allocator_traits<allocator_type>::rebind_traits<U>::destroy` (19.10.9.2), where `U` is either `allocator_type::value_type` or an internal type used by the container. These functions are called only for the container's element type, not for internal types used by the container. [Note: This means, for example, that a node-based container might need to construct nodes containing aligned buffers and call `construct` to place the element into the buffer. — end note]

§ 7.4. [vector]

In [vector.overview] add the declaration for `insert_from_capacity`:

```

namespace std {
    template<class T, class Allocator = allocator<T>>
    class vector {

        ...

        // 21.3.11.4, data access
        T* data() noexcept;
        const T* data() const noexcept;
        T* uninitialized_data() noexcept;

        // 21.3.11.5, modifiers
        template<class... Args> reference emplace_back(Args&&... args);
        void push_back(const T& x);
        void push_back(T&& x);
        void pop_back();

        template<class... Args> iterator emplace(const_iterator position, Args
        iterator insert(const_iterator position, const T& x);
        iterator insert(const_iterator position, T&& x);
        iterator insert(const_iterator position, size_type n, const T& x);
        template<class InputIterator>
            iterator insert(const_iterator position, InputIterator first, InputI
        iterator insert(const_iterator position, initializer_list<T> il);
        iterator insert_from_capacity(size_type n);
        iterator erase(const_iterator position);
        iterator erase(const_iterator first, const_iterator last);

        ...
    }
}

```

In [**vector.data**] add new p3-5:

```
T*      data() noexcept;
const T* data() const noexcept;
```

1. *Returns:* A pointer such that `[data(), data() + size())` is a valid range. For a non-empty vector, `data() == addressof(front())`.
2. *Complexity:* Constant time.

```
T*      uninitialized_data() noexcept;
```

3. *Returns:* A pointer to uninitialized storage that would hold elements in the range `[size(), capacity())`. [Note: This storage may be initialized through a pointer obtained by casting `T*` to `void*` and then to `char*`, `unsigned char*`, or `std::byte*`. ([basic.life]p6.4). - end note]
4. *Remarks:* This member function does not participate in overload resolution if `allocator_traits<Allocator>::rebind_traits<U>::implicit_construct(U*)` is not well-formed.
5. *Complexity:* Constant time.

In **[vector.modifiers]** add new p3-6:

2. *Complexity*: The complexity is linear in the number of elements inserted plus the distance to the end of the vector.

iterator insert_from_capacity(size_type n);

3. *Requires*: - $n \leq \text{capacity}() - \text{size}()$.
4. *Remarks*: - Appends n elements by implicitly creating them from capacity. The application must have initialized the storage backing these elements otherwise the behavior is undefined. This member function does not participate in overload resolution if `allocator_traits<Allocator>::rebind_traits<U>::implicit_construct(U*)` is not well-formed.
5. *Returns*: - an iterator to the first element inserted, otherwise `end()`.
6. *Complexity*: - The complexity is linear in the number of elements inserted. [Note: For some allocators, including the default allocator, actual complexity is constant time. - end note]

```
iterator erase(const_iterator position);
iterator erase(const_iterator first, const_iterator last);
void pop_back();
```

7. *Effects*: Invalidates iterators and references at or after the point of the erase.
8. ...

§ 8. Acknowledgments

Special thanks go to **Glen Fernandes** and **Mathias Stearn** for early comments and help with wording. Thanks also for early comments from **Nevin Liber**, **Agustín Bergé**, and **Arthur O'Dwyer** provided guidance on object lifetime and allocator interactions.

§ References

§ Informative References

[N4762]

Working Draft, Standard for Programming Language C++. 7 July 2018. URL:
<https://wg21.link/N4762>

[P0593R2]

Richard Smith. [Implicit creation of objects for low-level object manipulation](#). 11 February 2018.

URL: <https://wg21.link/p0593r2>

[P0653R2]

Glen Joseph Fernandes. [Utility to convert a pointer to a raw pointer](#). 9 November 2017. URL:

<https://wg21.link/p0653r2>

[P1020R0]

Glen Joseph Fernandes; Peter Dimov. [Smart pointer creation functions for default initialization](#).

May 2018. URL: <https://wg21.link/P1020R0>

[P1072R1]

Chris Kennelly; Mark Zeren. [Default Initialization for basic_string](#). October 2018. URL:

<https://wg21.link/P1071R1>